Ph.D. in Computer Science (DCS)

University of St.Gallen

Semantic-Aware Foundations for Theorem Proving

Research Proposal

Advisor:
Prof. Dr. Guido Salvaneschi
Institute of Computer Science
University of St.Gallen

PhD Student:
Jahrim Gabriele Cesario

Co-Advisor(s):
Prof. Dr. Max Willsey
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley

July 30, 2026

Abstract

Software verification is the discipline of establishing that a program meets its specification and is indispensable to improve our understanding of software systems and ensure their reliability. Theorem proving pursues verification by constructing formal machine-checkable proofs of correctness, in a logic that can capture program semantics. Constructing such proofs turns repeatedly on deciding whether terms are equal, and many theorem provers rest on a common foundation for equational reasoning. Reasoning in verification, however, is rarely about equality alone. Proofs split into cases, argue under assumptions, and appeal to facts that hold only conditionally, but provers today capture this richer semantics around that foundation rather than within it. This proposal argues that the foundation itself should reason beyond equality, letting provers do more of their work at the level they already share.

We pursue this idea for the e-graph, the equality-reasoning data structure at the heart of many provers. Extending it to capture a broader class of relationships between terms lets provers reason about them natively and opens room for optimizations, since one enriched structure can reuse work that separate encodings would repeat. Because the e-graph underlies so many provers, these gains extend well beyond any single tool: we consolidate the extensions into *Smart E-Graphs*, a single artifact that integrates with existing provers and, through them, reaches the broader verification ecosystem.

1 Introduction

Software verification is the discipline of assuring that a software system conforms to a given specification. It is a traditional area of research in computer science, drawing on both *software engineering* and *programming languages*. *Dynamic verification*, such as *testing* [2] and *fuzzing* [60], executes the system under selected configurations to discover deviations from the specification, i.e., *bugs*. Conversely, *static verification*, such as *model checking* [8] and *theorem proving* [15, 22], analyzes all possible behaviors of the system to guarantee that it always satisfies the specification. Since this is rarely feasible for complex systems, static verification often relies on *abstractions* that approximate the system and the specification to make the analysis tractable. In that sense, it is *static* because the analysis is performed without executing the original system.

In this proposal, we focus on *static verification* and specifically on *theorem proving* for software verification. A theorem prover is a tool that checks the validity of *formulas* in some *logic* e.g., whether a *goal* property (i.e., some behavior) is implied from a set of known facts (i.e., the system). To do so, theorem provers can apply a mix of two approaches [22]: the *model-theoretic* approach cleverly enumerates all possible models of the facts and checks if the goal holds, i.e., searches for the first system configuration that invalidates the expected behavior; the *proof-theoretic* approach derives the goal from the known facts using the set of valid inference rules of the logic, i.e., transforms the system into a logically equivalent system for which the property trivially holds.

The guarantees provided by static verification are essential to determine which conditions are *sufficient* or *necessary* for a system to behave as expected. For example, a *type system* like Rust's enforces *ownership semantics* in a program, which is a sufficient condition for memory safety and data race freedom [19]. More expressive analyzers, such as the Viper *verification language* [34] can check that given *preconditions* are sufficient to guarantee arbitrary *postconditions* of a program, based on *separation logic* [42] and *SMT solvers* [32]. At the boundary of expressiveness, *proof assistants* like Lean [33] can check the proofs of arbitrary properties of the system using *dependent type theory* [30, 9], but they enforce *totality* of definitions, which is a necessary condition for the soundness of the proofs. Together, these tools provide us with a better understanding of the system before it is deployed in the real world, preventing costly failures [27, 14, 28, 13, 29, 20]. Their relevance is even more pronounced with the surge in software produced by *generative artificial intelligence*, which has fueled a push to certify machine-generated artifacts from both academy [47, 56, 46, 52, 40, 36, 35] and industry [1, 53, 41, 24].

Theorem proving faces two fundamental challenges: *scalability* and *usability*. Scalability concerns the computational cost of the analysis: the space of models to enumerate or proofs to derive grows

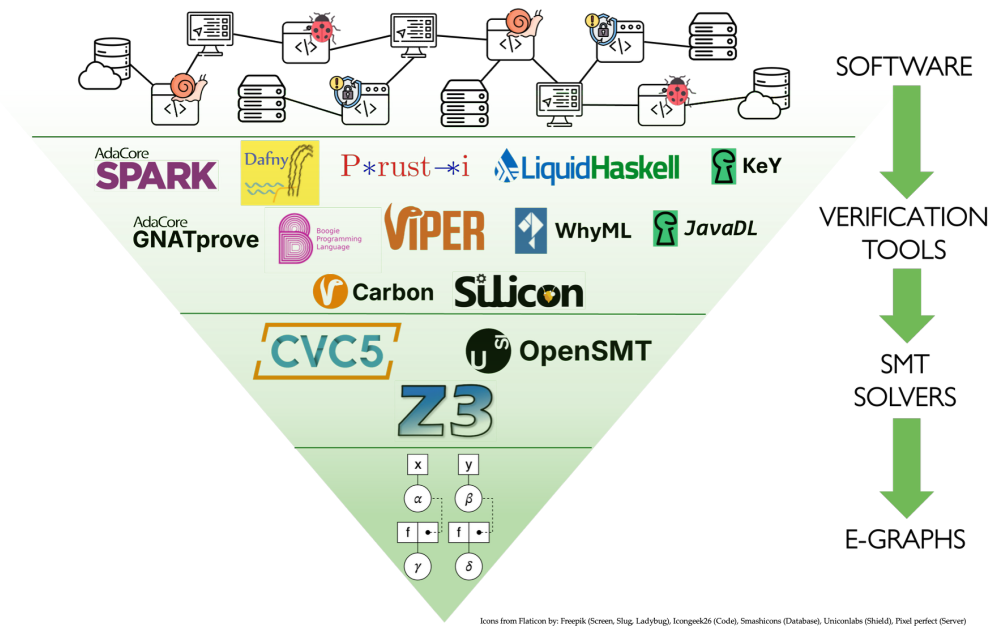

Figure 1: The static verification ecosystem based on SMT solvers and e-graphs.

rapidly, and is often undecidable, so a prover may fail to establish a goal within any practical time and memory budget. Usability concerns the human cost: expressing a system and its specification in a formal logic, and steering the prover toward a proof, demands considerable expertise and manual effort. These two challenges are in tension, and theorem provers are commonly classified by how they trade one against the other, according to their level of automation [16, 54]. *Automated theorem provers* (ATPs) require no human guidance and thus minimize the usability burden, but their automation reaches only a limited class of goals before the search becomes intractable; their central challenge is therefore scalability. *Interactive theorem provers* (ITPs) admit human guidance and can, in principle, prove arbitrary properties, so expressiveness is not the wall; their central challenge is instead the human effort this guidance requires, i.e., usability. In practice the two families are not sealed off: ITPs increasingly embed automation to ease this burden, through *tactics* that discharge routine subgoals and *hammers* that hand goals to automated provers [6].

Underlying much of this ecosystem is a single data structure: the *e-graph*, which compactly represents a congruence relation over terms and originates in the congruence-closure decision procedures of Nelson and Oppen [37]. Both families of provers reduce core reasoning tasks to operations on e-graphs (Figure 1), though to different ends. ATPs use them for *equational reasoning*: SMT solvers rely on congruence closure and e-matching to combine theories and instantiate quantifiers [31, 32], while *equality-saturation* engines make the e-graph the central representation for applying rewrites without committing to an order [51, 55]. ITPs use them for *automated proof search*: tactics such as the `grind` tactic in Lean discharge goals by saturating an e-graph with the available facts and their congruence consequences [25]. Because the same structure sits beneath so many otherwise disconnected tools, an improvement to the e-graph itself can propagate as an optimization across the whole ecosystem.

At its core, this proposal extends the e-graph beyond the equalities it was designed to represent, enriching it into a structure that reasons about a broader class of relationships between terms. We start from one of the most common patterns in theorem proving, *proof by cases*, and show how it can be made first-class in the e-graph. A proof by cases splits the goal into multiple subgoals, proven under mutually exclusive assumptions. Two ingredients are essential to this technique: *disequality reasoning*, which tracks disequalities arising from mutually exclusive cases, and *conditional reasoning*, which tracks which facts hold under which assumptions. With disequality reasoning, contradictions can be detected early, which can be exploited to prune the search space or in proofs by contradiction. With conditional reasoning, the entailment of conditions can be mapped to a dependency relation among e-graphs, which can be exploited for *lazy reasoning*, deferring computation until a query demands it, and for *parallel reasoning*, scheduling proof exploration across independent tasks.

Because these extensions live in the e-graph, the substrate shared across the ecosystem, their benefits are not confined to a single tool but reach any prover built on it. The gains fall along the two axes that divide the field. For ATPs, detecting contradictions early, deferring computation until it is needed, and exploring independent branches in parallel all attack the combinatorial blow-up of the search, addressing the scalability that most limits them. For ITPs, absorbing disequalities and conditional facts into the e-graph lets automation discharge goals that would otherwise fall to the user, addressing the usability that most limits them. Made once at the foundation, this improvement is inherited by every tool above it.

2 Background

This section reviews the background that motivates our proposal and the prior work it builds on. We begin with the landscape of theorem provers, identifying which of them stand to benefit from our contributions, and then describe the data structures that will be extended.

2.1 Theorem Provers

Automated Theorem Provers. *Automated theorem provers* discharge goals without human guidance, and how much each stands to gain from our work depends on how centrally it relies on e-graphs. *SMT solvers* such as Z3 [32] and cvc5 [3] use congruence closure and e-matching as a core theory solver, so extensions to the e-graph reach them directly. *Verification languages* such as Dafny [26], Viper [34], and F* [48] compile programs and their specifications to SMT queries, and so benefit indirectly, through their solver backend. *Equality-saturation* tools, including egg [55] and the inductive provers TheSy [45] and CCLemma [23], make the e-graph the central representation for exploring rewrites, and stand to gain the most. By contrast, *saturation-based* superposition provers such as Vampire [21] and E [43] organize their search around term indexing and ordered rewriting rather than a shared e-graph, and are the least likely to benefit from this line of work.

Interactive Theorem Provers. *Interactive theorem provers* let a user steer the proof while the tool checks each step. Their kernels rest on rich type theories or higher-order logics, as in Isabelle [38], Coq [5], Agda [39], and Lean [33], and this core proof checking does not use e-graphs, so our extensions have little to offer it. Their proof automation is another matter: e-graph-based tactics such as Lean's `grind` [25] and Coq's congruence closure [10] discharge goals by saturating an e-graph with the available facts, and it is these components, rather than the kernels, that our work stands to improve.

2.2 Equational Reasoning

Union-Find. A *union-find*, or disjoint-set structure [50], maintains an *equivalence relation* over a set of elements using two operations: *find* returns a canonical representative of the class containing an element, so that two elements are equivalent exactly when they share a representative, and *union* merges two classes by making one representative point to the other. With union by rank and path compression, both run in near-constant amortized time [49]. The structure thus maintains precisely the equivalence relation generated by the equalities asserted so far, that is, their reflexive, symmetric, and transitive closure, but records nothing about the structure of the terms that name the elements.

E-Graphs. An *e-graph* [37, 55] lifts this idea from opaque elements to functional *terms*, maintaining not merely an equivalence relation but a *congruent* one: a relation additionally closed under congruence, so that whenever the arguments of a function are pairwise equivalent, the applications are too. A term is stored as an *e-node*, a function symbol applied to a tuple of child *e-classes*, and an e-class is a set of e-nodes that have been proven equal. Internally, an e-graph is a union-find over e-class identifiers together with a hash-cons mapping each e-node to its e-class, so *find* and *union* carry over unchanged. Merging two e-classes with *union*, however, can break the congruence invariant, since two e-nodes that have just become congruent may still sit in different e-classes. For example, suppose an e-graph holds the terms $f(a)$ and $f(b)$ in distinct e-classes; asserting $a = b$ merges the classes of a

and b, so congruence now demands $f(a) = f(b)$, yet the two applications still sit in separate e-classes until the invariant is restored. The *rebuild* operation restores the invariant by repeatedly merging such newly congruent e-nodes until a fixpoint is reached [55]. An e-graph therefore represents the *congruent equivalence relation* generated by the asserted equalities, that is, their reflexive, symmetric, transitive, and congruent closure [59].

Equality Saturation. *Equality saturation* [51, 55] is a framework built around e-graphs that repeatedly applies a set of rewrite rules, non-destructively so that rules need not be ordered, until it is *saturated* or a resource bound is reached. Then, one can *extract* an optimal representative under a cost model or prove two terms equal under the axioms of the rewrite rules. In detail, each rewrite $\ell \to r$ is applied by *e-matching*, finding all instances of the pattern ℓ up to the equivalences the e-graph records [31] and making r equal to every match. Sidestepping the *phase-ordering* problem this way, equality saturation underlies applications from compiler optimization [51] to inductive theorem proving, as in TheSy [45] and CCLemma [23], while the same e-matching mechanism drives quantifier instantiation in SMT solvers [31].

E-Graph Extensions. E-graphs have already been extended in several directions. Contextual variants carry several congruent equivalence relations at once, embedded in the language as *assume nodes* [11] or in the structure as *colors* in Easter Egg [44]. Parallelism has been studied beneath the e-graph, in concurrent union-finds [18], and around it, where equality saturation has been parallelized with thread-safe structures and deferred, batched updates [12]. This proposal takes the structural route to e-graph extensions, which leaves the term space untouched, and pursues parallelism as the exploration of many relations under different assumptions rather than the saturation of a single one.

3 Proposal

This proposal extends the e-graph with semantic information beyond the equalities it was designed to represent, into a structure that reasons about a broader class of relationships between terms. Taking *proof by cases* as our guide, we make disequalities and conditional facts first-class citizens of the e-graph and exploit the resulting structure for optimization. Since the e-graph is the shared substrate of much of the theorem-proving ecosystem, these extensions stand to benefit automated and interactive provers alike.

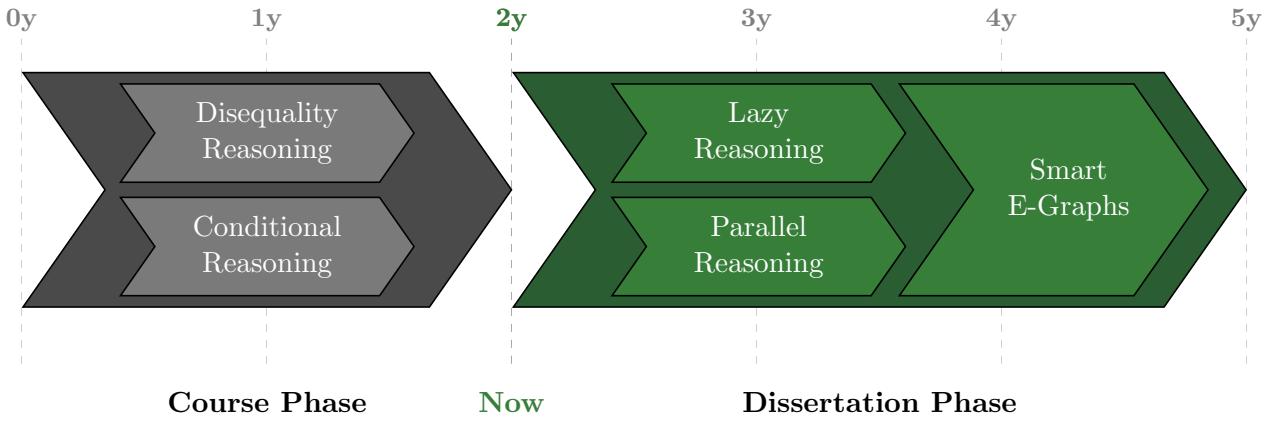

Figure 2: High-level plan for the Ph.D. proposal.

Figure 2 outlines the high-level plan for at most five years of the Ph.D., split into two phases. During the *course phase*, in the first two years, we developed the two *semantic* extensions that enrich what the e-graph represents: disequality reasoning and conditional reasoning. During the *dissertation phase*, we pursue the *optimizations* that these semantics make possible: lazy reasoning and parallel reasoning. During the development, the two lines converge into one final deliverable, *Smart E-Graphs*,

which we will integrate into both automated and interactive provers to demonstrate the benefits of our work.

4 Course Phase

In this section, we detail the *course phase* of the proposal, during which we developed two semantic extensions that enrich the reasoning capabilities of classical e-graphs: *disequality reasoning* and *conditional reasoning*.

4.1 Disequality Reasoning

First, I contributed to *dis/equality graphs*, which extend e-graphs with direct support for disequality [59]. Equality is represented as usual, while a disequality is stored as a first-class, undirected *disequality edge* between two e-classes. This contrasts with the *embedding* techniques prevalent in SMT solvers and verifiers, which reify (dis)equality into the term language, using a symbol such as `eq` or `ne` and saturating it against `false`. Embeddings enlarge the e-graph with extra e-nodes and e-classes, or require saturation; a disequality edge only requires storing a pair of e-classes plus some ad-hoc bookkeeping without affecting the term space, reducing both time and memory overhead.

We studied this design formally and empirically. The work introduced a formal framework for reasoning about e-graphs and used it to prove that an e-graph coincides with the reflexive, symmetric, transitive, and congruent closure of the asserted equalities. Generalizing to dis/equality graphs, we proved that they detect any contradiction between the recorded equalities and the asserted disequalities, and we gave an analytical bound showing them to be asymptotically more efficient than the embeddings. We implemented them as an extension to egg [55] and evaluated them in an EUF solver and the Propel automated theorem prover [57, 58] on standard benchmarks, where direct support for disequalities outperformed embedding-based encodings, in line with the analytical result.

4.2 Conditional Reasoning

Second, I authored *versioned e-graphs*, which extend e-graphs to represent equalities that hold only conditionally [7]. Under proof by cases, and branching settings more generally, each branch assumes a different set of equalities on top of a shared base. A traditional e-graph encodes a single global equality relation, so these contextual equalities are handled by keeping one e-graph per branch, which replicates the equalities the branches share. A versioned e-graph instead encodes many equivalence sets, one per *version*, in a single structure that maximizes sharing across them, resting on a *versioned union-find* that records the equalities common to several versions once.

As before, we studied this design formally and empirically. The work formalized versioned union-finds and versioned e-graphs and proved them correct, and analyzed the amortized time and space complexity of the versioned union-find against the cloning of a separate e-graph per branch. We realized the design in Veg, a standalone Rust library inspired by egg [55] and evaluated it against cloning, a variant based on persistent data structures, and the state of the art Easter Egg [44] across two case studies, an EUF solver and the TheSy theorem prover [45], where versioned e-graphs used up to 30% less memory and ran up to 4× faster, with the gap widening as the number of explored terms and branches grew.

5 Dissertation Phase

In this section, we detail the *dissertation phase* of the proposal, in which we pursue the two optimizations that the course-phase semantics make possible: *lazy reasoning* and *parallel reasoning*, which converge into *Smart E-Graphs*, the deliverable of the proposal.

5.1 Lazy Reasoning

With a single structure carrying one equivalence relation per assumption context, lazy reasoning asks how to exploit the dependencies between these contexts to avoid redundant work: the conditions that guard them are logically related, so the relation induced by one often determines much of the relation induced by another. As these relations form a lattice, two operations arise alongside the *fork* that opens a context: a *meet*, the closure of their union, which merges facts from different subgoals; and a *join*, their intersection, which keeps the facts common to every case of a split. The fork alone, a specific form of laziness that in effect clones a relation on demand, is what versioned e-graphs already provide; the goal now is to exploit richer dependencies between contexts, such as those induced by the semantics of the conditions that separate them, e.g., so that reasoning under a condition like $a = b \to c = d$ can reuse the relation already computed under $a = b$. Laziness reifies an inherited equality only when queried, amortizing the cost over the queries that use it. We can evaluate the operations in isolation, against eager counterparts, and end to end inside one of the aforementioned provers, where case splits arise in proofs by cases, or an SMT solver, where case-splitting paradigms such as cube-and-conquer [17] are a natural workload. In either setting we draw on established benchmarks: the suite of the host prover for the former, and the SMT-LIB benchmark library [4] for the latter, restricted to the divisions where case splitting dominates the search.

5.2 Parallel Reasoning

Parallel reasoning treats the lattice of equivalence relations as a dependency graph of tasks, each exploring one relation modulo its dependencies, and asks how to schedule that graph across many workers. Three difficulties shape the problem. No task is guaranteed to terminate, since saturating a context need not reach a fixpoint, so execution must be incremental, each task propagating partial results for downstream tasks to build on rather than running to completion. The lattice itself may be unbounded, as the conditions worth exploring are open-ended, so scheduling must interleave advancing the known relations with discovering new contexts, each discovery triggering a further scheduling round. And because the tasks share one underlying structure, a good schedule must keep synchronization and contention low. Addressing these statically where the lattice is known, and dynamically where it is not, is the crux of this direction. We can evaluate a schedule by its speed-up and scalability as the number of threads grows, relative to sequential exploration and to an ideal parallelization of the same dependency graph, drawing on the same benchmarks as in Section 5.1.

5.3 Smart E-Graphs

Smart E-Graphs is the deliverable of the proposal: one e-graph library with all four proposed extensions. A host prover can adopt it in place of its own structure and enable only the extensions it needs, so that our artifact can reach the verification ecosystem rather than one tool. Our extensions can interact in subtle ways, so their composition is non-trivial. For example, disequalities should be interpreted per version and be preserved by the meet and join. We will develop *Smart E-Graphs* incrementally, validating it against the benchmarks above in the provers our prior work already targets, and release the artifact open source.

References

[1] Amazon Web Services. *Prevent Factual Errors from LLM Hallucinations with Mathematically Sound Automated Reasoning Checks*. Amazon Web Services. 2024. URL: `https://aws.amazon.com/blogs/aws/prevent-factual-errors-from-llm-hallucinations-with-mathematically-sound-automated-reasoning-checks-preview/` (visited on 07/23/2026).

[2] Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. 2nd ed. Cambridge University Press, 2016. DOI: `10.1017/9781316771273`.

[3] Haniel Barbosa et al. “cvc5: A Versatile and Industrial-Strength SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 13243. LNCS. Springer, 2022, pp. 415–442. DOI: 10.1007/978-3-030-99524-9_24.

[4] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. *The Satisfiability Modulo Theories Library (SMT-LIB)*. www.SMT-LIB.org. 2016.

[5] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development. Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer, 2004. DOI: 10.1007/978-3-662-07964-5.

[6] Jasmin Christian Blanchette et al. “Hammering towards QED”. In: *Journal of Formalized Reasoning* 9.1 (2016), pp. 101–148. DOI: 10.6092/issn.1972-5787/4593.

[7] Jahrim Gabriele Cesario et al. “Versioned E-Graphs”. In: *Proceedings of the ACM on Programming Languages* 10.PLDI (2026). DOI: 10.1145/3808249.

[8] Edmund M. Clarke et al., eds. *Handbook of Model Checking*. Springer, 2018. DOI: 10.1007/978-3-319-10575-8.

[9] Thierry Coquand and Gérard Huet. “The Calculus of Constructions”. In: *Information and Computation* 76.2–3 (1988), pp. 95–120. DOI: 10.1016/0890-5401(88)90005-3.

[10] Pierre Corbineau. “Deciding Equality in the Constructor Theory”. In: *Types for Proofs and Programs (TYPES)*. Ed. by Thorsten Altenkirch and Conor McBride. Vol. 4502. LNCS. Springer, 2007, pp. 78–92. DOI: 10.1007/978-3-540-74464-1_6.

[11] Samuel Coward, Theo Drane, and George A. Constantinides. “Constraint-Aware E-Graph Rewriting for Hardware Performance Optimization”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024). DOI: 10.1109/TCAD.2024.3483096.

[12] Jonathan Van der Cruysse et al. “Parallel and Customizable Equality Saturation”. In: *Proceedings of the 35th ACM SIGPLAN International Conference on Compiler Construction (CC)*. ACM, 2026, pp. 94–105. DOI: 10.1145/3771775.3786266.

[13] Zakir Durumeric et al. “The Matter of Heartbleed”. In: *Proceedings of the 2014 Internet Measurement Conference (IMC)*. 2014, pp. 475–488. DOI: 10.1145/2663716.2663755.

[14] Alan Edelman. “The Mathematics of the Pentium Division Bug”. In: *SIAM Review* 39.1 (1997), pp. 54–67. DOI: 10.1137/S0036144595293959.

[15] John Harrison. *Handbook of Practical Logic and Automated Reasoning*. Cambridge University Press, 2009. DOI: 10.1017/CBO9780511576430.

[16] John Harrison, Josef Urban, and Freek Wiedijk. “History of Interactive Theorem Proving”. In: *Computational Logic*. Ed. by Jörg H. Siekmann. Vol. 9. Handbook of the History of Logic. Elsevier, 2014, pp. 135–214.

[17] Marijn J. H. Heule et al. “Cube and Conquer: Guiding CDCL SAT Solvers by Lookaheads”. In: *Hardware and Software: Verification and Testing (HVC)*. Vol. 7261. Lecture Notes in Computer Science. Springer, 2012, pp. 50–65. DOI: 10.1007/978-3-642-34188-5_8.

[18] Siddhartha V. Jayanti and Robert E. Tarjan. “A Randomized Concurrent Algorithm for Disjoint Set Union”. In: *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2016. DOI: 10.1145/2933057.2933108.

[19] Ralf Jung et al. “RustBelt: Securing the Foundations of the Rust Programming Language”. In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018). DOI: 10.1145/3158154.

[20] Paul Kocher et al. “Spectre Attacks: Exploiting Speculative Execution”. In: *2019 IEEE Symposium on Security and Privacy (S&P)*. 2019, pp. 1–19. DOI: 10.1109/SP.2019.00002.

[21] Laura Kovács and Andrei Voronkov. “First-Order Theorem Proving and Vampire”. In: *Computer Aided Verification (CAV)*. Vol. 8044. LNCS. Springer, 2013, pp. 1–35. DOI: 10.1007/978-3-642-39799-8_1.

[22] Daniel Kroening and Ofer Strichman. *Decision Procedures: An Algorithmic Point of View*. 2nd ed. Texts in Theoretical Computer Science. Springer, 2016. DOI: 10.1007/978-3-662-50497-0.

[23] Cole Kurashige et al. "CCLemma: E-Graph Guided Lemma Discovery for Inductive Equational Proofs". In: *Proceedings of the ACM on Programming Languages* 8.ICFP (2024). DOI: 10.1145/3674653.

[24] Guillaume Lample et al. "HyperTree Proof Search for Neural Theorem Proving". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022.

[25] Lean FRO. *The `grind` Tactic*. The Lean Language Reference. 2026. URL: https://lean-lang.org/doc/reference/latest/The--grind--tactic/ (visited on 07/20/2026).

[26] K. Rustan M. Leino. "Dafny: An Automatic Program Verifier for Functional Correctness". In: *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR)*. Vol. 6355. LNCS. Springer, 2010, pp. 348–370. DOI: 10.1007/978-3-642-17511-4_20.

[27] Nancy G. Leveson and Clark S. Turner. "An Investigation of the Therac-25 Accidents". In: *Computer* 26.7 (1993), pp. 18–41. DOI: 10.1109/MC.1993.274940.

[28] Jacques-Louis Lions. *Ariane 5 Flight 501 Failure: Report by the Inquiry Board*. Tech. rep. European Space Agency, 1996.

[29] Moritz Lipp et al. "Meltdown: Reading Kernel Memory from User Space". In: *27th USENIX Security Symposium (USENIX Security)*. 2018, pp. 973–990.

[30] Per Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.

[31] Leonardo de Moura and Nikolaj Bjørner. "Efficient E-Matching for SMT Solvers". In: *Automated Deduction (CADE)*. Vol. 4603. LNCS. Springer, 2007, pp. 183–198. DOI: 10.1007/978-3-540-73595-3_13.

[32] Leonardo de Moura and Nikolaj Bjørner. "Z3: An Efficient SMT Solver". In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 4963. LNCS. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24.

[33] Leonardo de Moura and Sebastian Ullrich. "The Lean 4 Theorem Prover and Programming Language". In: *Automated Deduction (CADE)*. Vol. 12699. LNCS. Springer, 2021, pp. 625–635. DOI: 10.1007/978-3-030-79876-5_37.

[34] Peter Müller, Malte Schwerhoff, and Alexander J. Summers. "Viper: A Verification Infrastructure for Permission-Based Reasoning". In: *Verification, Model Checking, and Abstract Interpretation (VMCAI)*. Vol. 9583. LNCS. Springer, 2016, pp. 41–62. DOI: 10.1007/978-3-662-49122-5_2.

[35] Niels Mündler et al. "Type-Constrained Code Generation with Language Models". In: *Proceedings of the ACM on Programming Languages* 9.PLDI (2025), pp. 601–626. DOI: 10.1145/3729274.

[36] Shaan Nagy et al. "ChopChop: A Programmable Framework for Semantically Constraining the Output of Language Models". In: *Proceedings of the ACM on Programming Languages* 10.POPL (2026). DOI: 10.1145/3776708.

[37] Greg Nelson and Derek C. Oppen. "Fast Decision Procedures Based on Congruence Closure". In: *Journal of the ACM* 27.2 (1980), pp. 356–364. DOI: 10.1145/322186.322198.

[38] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Vol. 2283. LNCS. Springer, 2002. DOI: 10.1007/3-540-45949-9.

[39] Ulf Norell. "Dependently Typed Programming in Agda". In: *Advanced Functional Programming (AFP)*. Vol. 5832. LNCS. Springer, 2009. DOI: 10.1007/978-3-642-04652-0_5.

[40] Gabriel Poesia et al. "Synchromesh: Reliable Code Generation from Pre-trained Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022.

[41] Stanislas Polu et al. "Formal Mathematics Statement Curriculum Learning". In: *International Conference on Machine Learning (ICML)*. 2022.

[42] John C. Reynolds. "Separation Logic: A Logic for Shared Mutable Data Structures". In: *Logic in Computer Science (LICS)*. IEEE, 2002, pp. 55–74. DOI: 10.1109/LICS.2002.1029817.

[43] Stephan Schulz. "System Description: E 1.8". In: *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR)*. Vol. 8312. LNCS. Springer, 2013, pp. 735–743. DOI: 10.1007/978-3-642-45221-5_49.

[44] Eytan Singher and Shachar Itzhaky. "Easter Egg: Equality Reasoning Based on E-Graphs with Multiple Assumptions". In: *Proceedings of the 24th Conference on Formal Methods in Computer-Aided Design (FMCAD)*. TU Wien Academic Press, 2024, pp. 70–83. DOI: 10.34727/2024/isbn.978-3-85448-065-5_13.

[45] Eytan Singher and Shachar Itzhaky. "Theory Exploration Powered by Deductive Synthesis". In: *Computer Aided Verification (CAV)*. Vol. 12760. LNCS. Springer, 2021. DOI: 10.1007/978-3-030-81688-9_6.

[46] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. "Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean". In: *International Conference on Neuro-Symbolic Systems (NeuS)*. 2025.

[47] Chuyue Sun et al. "Clover: Closed-Loop Verifiable Code Generation". In: *AI Verification (SAIV)*. Vol. 14846. LNCS. Springer, 2024, pp. 134–155. DOI: 10.1007/978-3-031-65112-0_7.

[48] Nikhil Swamy et al. "Dependent Types and Multi-Monadic Effects in F*". In: *Principles of Programming Languages (POPL)*. ACM, 2016, pp. 256–270. DOI: 10.1145/2914770.2837655.

[49] Robert E. Tarjan and Jan van Leeuwen. "Worst-Case Analysis of Set Union Algorithms". In: *Journal of the ACM* 31.2 (1984), pp. 245–281. DOI: 10.1145/62.2160.

[50] Robert Endre Tarjan. "Efficiency of a Good But Not Linear Set Union Algorithm". In: *Journal of the ACM* 22.2 (1975), pp. 215–225. DOI: 10.1145/321879.321884.

[51] Ross Tate et al. "Equality Saturation: A New Approach to Optimization". In: *Principles of Programming Languages (POPL)*. ACM, 2009, pp. 264–276. DOI: 10.1145/1480881.1480915.

[52] Amitayush Thakur et al. "CLEVER: A Curated Benchmark for Formally Verified Code Generation". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2025.

[53] Trieu H. Trinh et al. "Solving Olympiad Geometry without Human Demonstrations". In: *Nature* 625 (2024), pp. 476–482. DOI: 10.1038/s41586-023-06747-5.

[54] Freek Wiedijk, ed. *The Seventeen Provers of the World: Foreword by Dana S. Scott*. Vol. 3600. LNAI. Springer, 2006. DOI: 10.1007/11542384.

[55] Max Willsey et al. "egg: Fast and Extensible Equality Saturation". In: *Proceedings of the ACM on Programming Languages* 5.POPL (2021). DOI: 10.1145/3434304.

[56] Kaiyu Yang et al. "LeanDojo: Theorem Proving with Retrieval-Augmented Language Models". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.

[57] George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. "Automated Verification of Fundamental Algebraic Laws". In: *Proceedings of the ACM on Programming Languages* 8.PLDI (2024). DOI: 10.1145/3656408.

[58] George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. "Type-Checking CRDT Convergence". In: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023). DOI: 10.1145/3591276.

[59] George Zakhour et al. "Dis/Equality Graphs". In: *Proceedings of the ACM on Programming Languages* 9.POPL (2025), pp. 2282–2305. DOI: 10.1145/3704913.

[60] Andreas Zeller et al. *The Fuzzing Book*. CISPA Helmholtz Center for Information Security, 2024. URL: https://www.fuzzingbook.org/.